Traffic Signal Control with Cycle-Level DQN

Author: Snehita Satya Sri Kandula, Alejandra Rivas Montes, Naiji Zhai

(Authors listed alphabetically by surname)

Abstract

This paper presents a Deep Q-Network (DQN) approach to implement adaptive traffic signal control at a single intersection. The agent operates at a cycle-level, adjusting green-phase durations after every complete signal cycle rather than switching phases at each simulation step. The environment is a custom Gymnasium simulation with stochastic, time-varying vehicle arrivals and a four-phased fixed-cycle structure, modeled after Moreno-Malo et al. (2024). While the original work is about a multi-intersection SUMO environment, this project focuses on a simplified single-intersection environment to identify learning dynamics. The DQN agent is compared against six baseline policies including fixed-time, demand-aware, tuned-static, queue-based heuristic, and random policies. Ablation experiments were conducted to examine the effects of training duration, network size, and reward shaping on the agent's performance under both symmetric and asymmetric-peaky demand conditions. Maximum results are achieved under asymmetric-peaky demand, where DQN records a reward gain of 706 points and a 7.741% reduction in average waiting time compared to the best fixed-time baseline. Whereas, Under symmetric demand, gains are modest but consistently positive under stable hyperparameter settings, yielding approximately 1.0% to 1.3% wait reduction.

1. Introduction

Traffic signal control is a sequential decision problem: each signal decision changes future queues, delays, and discharge opportunities. Traditional fixed-time control assigns predetermined green durations to each phase regardless of current traffic conditions. While fixed-time policies are simple and robust, they cannot adapt to directional imbalance or time-varying demand. When one direction of the road is busier than the others, a fixed signal policy assigns green to empty lanes while the busier lanes remain stuck waiting.

Reinforcement learning provides a natural framework for adaptive traffic signal control. An agent interacts with a traffic environment, observes the current state of queues and flows, and selects timed actions to reduce congestion and wait time. Deep Q-Networks (DQN), introduced by Mnih et al. (2015) [2], combine Q-learning with deep neural networks, experience replay, and a target network to stabilize training. Their work has shown a human-level control on Atari games and established DQN as a foundational algorithm in deep reinforcement learning.

Moreno-Malo et al. (2024) [1] presented a distributed DQN agent system for traffic signal control at intersections simulated in SUMO, achieving a 44% reduction in wait times compared to fixed-time control. This project uses Deep Q-Networks (DQN) for discrete traffic-signal actions, following the core principle of value-based reinforcement learning in discrete action spaces [2]. The project's environment and control structure are intentionally modeled after Moreno-Malo et al. (2024) [1], specifically in its use of fixed four-phase cycle with safety phases, actions that adjust cycle timing rather than switching phases on a second-by-second basis, state features incorporating short-term traffic history, and comparison against fixed-time baselines.

This paper reports results from a systematic evaluation that covers hyperparameter pilot tuning, multi-seed ablations on training duration and network size, and a final difficulty comparison between symmetric and asymmetric-peaky demand conditions. The evaluation framework uses a tuned-static fixed-time baseline as the primary reference to ensure that any reported DQN improvement does not reflect an unfair comparison.

2. Methods

2.1 Environment Design

The environment simulates a single four-approach intersection with four lanes: North, South, East, and West. At each simulation step, vehicles arrive independently in each lane following Bernoulli process with time-varying arrival probabilities. The probabilities follow a sinusoidal demand wave to simulate rush-hour traffic patterns, with adjustable amplitude and period. Capacity of vehicle departure is set to two vehicles per second per green lane.

The signal cycle has four sequential phases: NS green, NS yellow, EW green, and EW yellow. Base durations are 10 seconds of green and 2 seconds of yellow per direction, producing a 24-second base cycle. The agent makes only a single decision for a complete cycle, at the cycle boundary. This avoids partial-cycle action overwriting and gives cleaner credit assignment.

2.2 Action Space

The agent chooses from three discrete actions at each decision step. Action 0 favors the NS direction, increasing the NS green phase by a configurable delta (default 2 seconds) and reducing EW green by the same amount. Action 1 keeps the base cycle unchanged. Action 2

favors the EW direction, decreasing NS green and increasing EW green. A minimum green time constraint of 6 seconds is enforced for each direction.

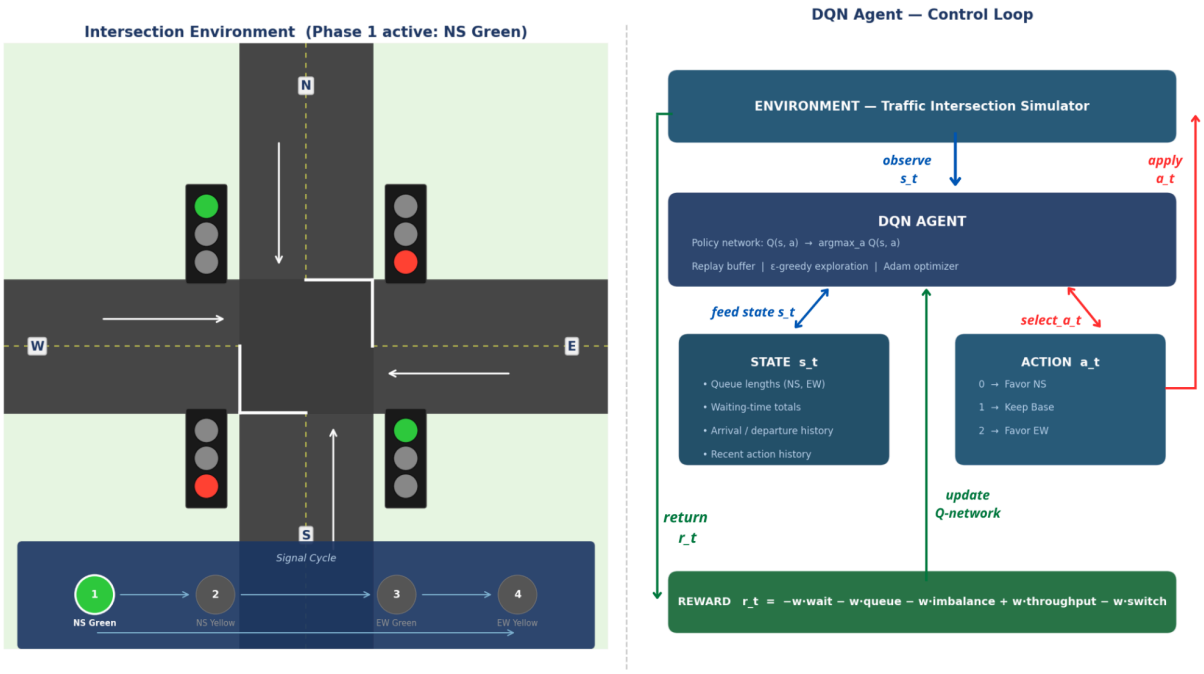


Figure 1: Environment and agent interaction diagram. (Left) Top-down view of the single-intersection environment showing the four-phase signal cycle (NS Green → NS Yellow → EW Green → EW Yellow) with active phase highlighted. (Right) DQN agent control loop: the agent observes the state s_t from the environment, selects an action a_t ∈ {0, 1, 2} via the learned policy Q(s, a), applies it to the environment, and receives a reward r_t used to update the Q-network.

2.3 Observation Space

The 36-dimensional observation vector includes the current queue lengths for all four lanes, cumulative waiting time totals for each lane, and a history of vehicle arrivals over the past K=3 cycles. It also includes a corresponding history of vehicle departures over the same K=3 cycles, along with the current phase identifier and the remaining time ratio of the active phase. In

addition, the vector contains the last action taken and the action taken two decision steps earlier. Before being passed to the network, all values are clipped and normalized to the range [0,1].

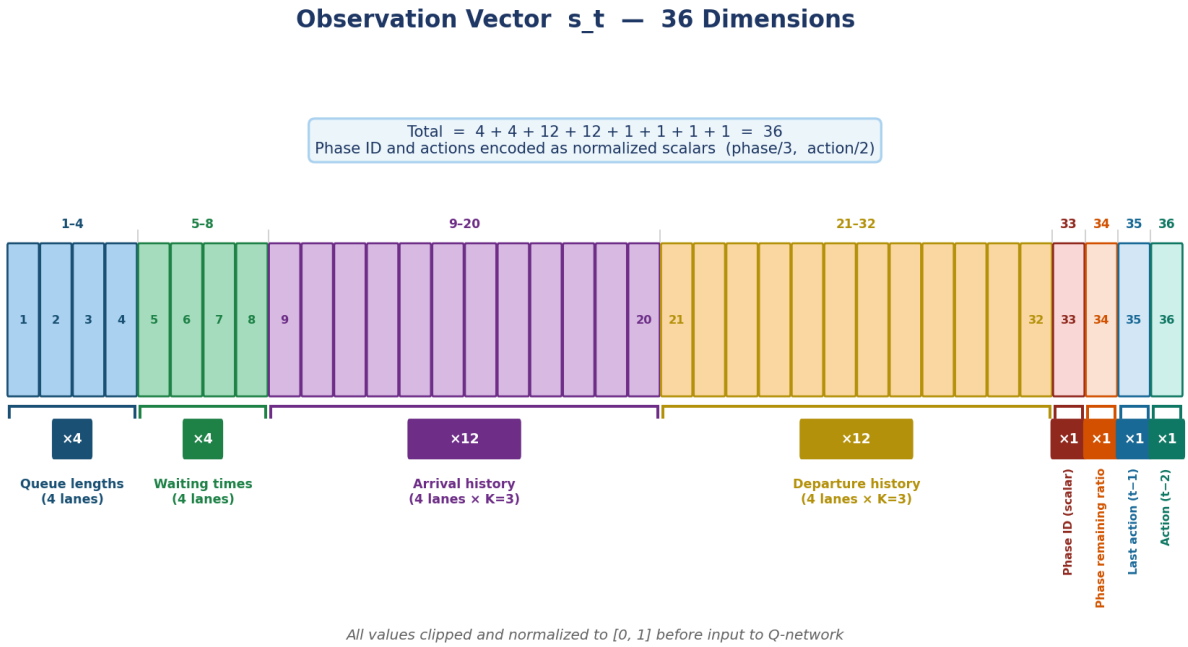


Figure 2: Structure of the 36-dimensional observation vector s_t.

2.4 Reward Function

The cycle-level reward is a weighted linear combination of five components computed over the duration of the most recent cycle. The reward penalizes mean waiting time with weight 1.0, mean queue length with weight 0.2, and mean NS-EW queue imbalance with weight 0.2. It rewards throughput, measured as the number of vehicles departed per cycle second, with weight 0.5, and penalizes unnecessary action changes between consecutive cycles with weight 0.3. Formally:

$$R = -1.0 \cdot wait - 0.2 \cdot queue - 0.2 \cdot imbalance + 0.5 \cdot throughput - 0.3 \cdot action_changed$$

This multi-objective structure directs the agent to lower the congestion and wait time while maintaining safe and stable signal operation.

2.5 DQN Agent

The DQN agent maintains an online Q-network and a target Q-network, both are fully connected neural networks using ReLU activations. The architecture used in all final experiments is: input (36) → 1024 → 512 → 64 → output (3 actions). The model-size ablation also evaluates a variant using a final hidden dimension 128 instead of 64. The target network's weights are updated from the online network for every 20 episodes. A replay buffer of 100,000 transitions with a mini-batch size of 64 is used for experience replay. The optimizer is Adam (Paszke et al., 2019 [3]) with a discount factor of gamma = 0.99.

Exploration follows an epsilon-greedy policy with epsilon decaying linearly from 1.0 to 0.05 over a configurable number of global steps. Depending on the ablation condition, the agent was trained for either 600 or 1800 episodes, where each episode consists of 300 decision steps.

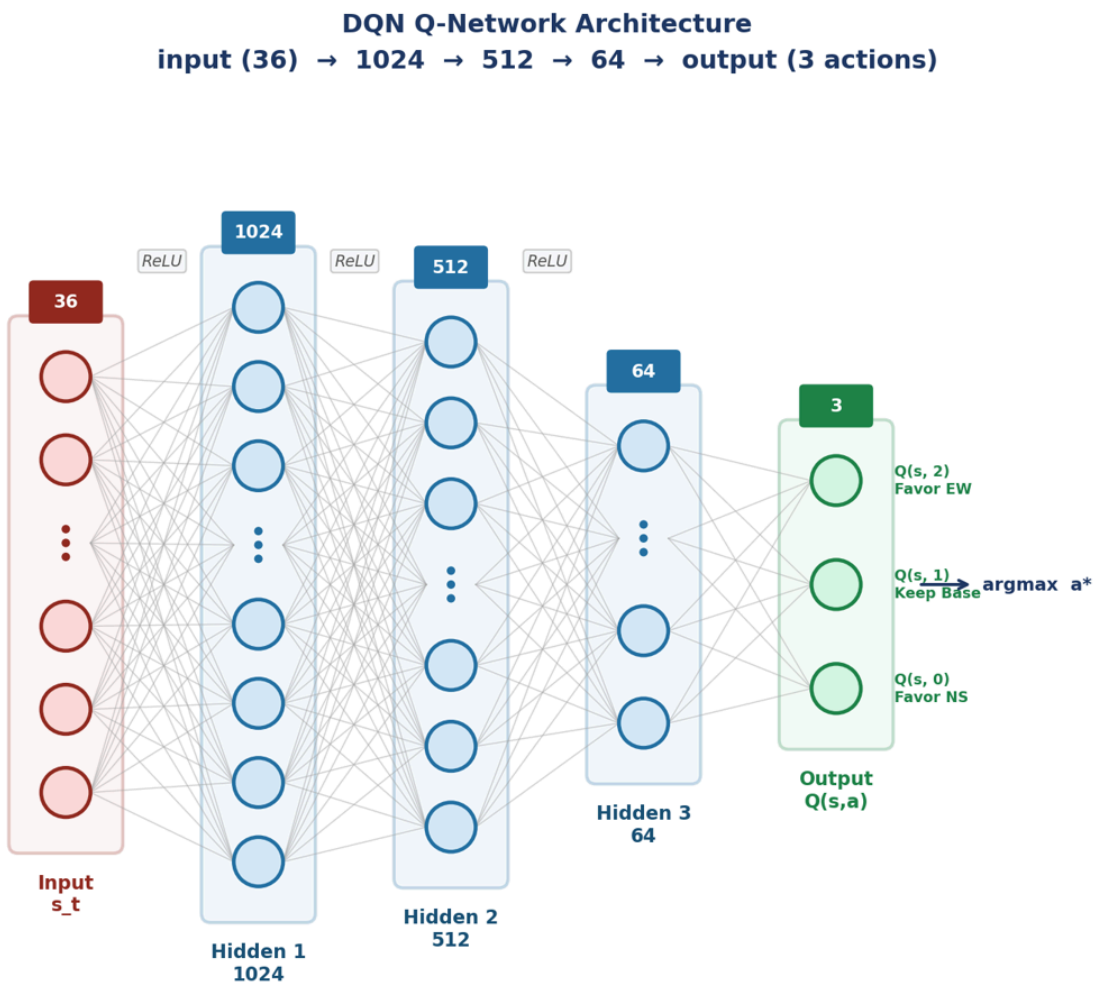


Figure 3: DQN Q-network architecture. The fully connected network receives the 36-dimensional observation vector s_t as input and processes it through three hidden layers of 1024, 512, and 64 units with ReLU activations. The output layer produces three Q-values — one per action — corresponding to favoring NS (action 0), maintaining the base cycle (action 1), and favoring EW (action 2). The selected action a is the argmax over these Q-values. Both the online network and the target network share this architecture; target weights are copied from the online network every 20 episodes.*

2.6 Baseline Policies

Six baseline policies are evaluated for comparison. The Fixed-Time (Keep) policy always selects action 1, keeping the base cycle unchanged without any adjustments. The Fixed-Time (Demand-Aware) policy chooses a single fixed action based on the relative NS and EW arrival probabilities at evaluation time, but remains static throughout each episode and does not adapt to changing conditions. The Fixed-Time (Tuned-Static) policy evaluates all three static actions and selects the one that achieves the highest average episode reward; this is used as the primary fairness baseline since it represents the best possible fixed-time performance. In addition, three Static policies fix the action permanently to 0, 1, or 2 respectively. The actions were selected uniformly at random by the Random policy. The Queue-Based Policy compares total NS and EW queue lengths and favors the more congested direction, using a deadband of 0.05 to avoid frequent switching.

2.7 Evaluation Metrics

The primary metrics used for comparing DQN against the tuned-static baseline are defined as follows. reward_gain_vs_tuned represents the difference in average episode reward between DQN and the tuned-static baseline, where higher values indicate better. wait_diff_vs_tuned is the absolute difference in average waiting time per simulation second, computed as the tuned-static wait minus the DQN wait, so that positive values indicate DQN reduces waiting. percent_wait_reduction_vs_tuned is the percentage reduction: (wait_tuned - wait_DQN) / wait_tuned × 100%. All multi-seed results are reported as mean ± standard deviation, with 95% confidence interval half-width equal to 1.96 × std / sqrt(n) shown in parentheses.

2.8 Experimental Protocol

The experimental protocol is organized into four stages. First, a single-seed hyperparameter pilot search is performed under asymmetric-peaky demand, evaluating two learning rates (1e-3 and 3e-4) and two epsilon decay values (360,000 and 600,000 steps). This stage is used only to identify a suitable configuration for later stages and is not used as final statistical evidence. Second, multi-seed ablation studies (n=4, seeds 42–45) are conducted under symmetric demand to analyze impacts of training duration (600 vs. 1800 episodes) and network size (hidden64 vs. hidden128). Third, a final difficulty evaluation (n=5, seeds 42–46) compared the selected configuration under symmetric demand versus asymmetric-peaky demand, with each seed evaluated on 50 episodes. Fourth, an additional symmetric ceiling sweep evaluated six hyperparameter configurations (n=5, seeds 42–46, 100 episodes per seed) to determine whether modest symmetric gains reflect a fundamental ceiling or hyperparameter sensitivity.

3. Results

3.1 Baseline Evaluation (Symmetric Demand, Single Run)

Table 2 and Figure 4 present the results of the pilot hyperparameter search under asymmetric-peaky demand. The configuration with lr = 1e-3 and epsilon_decay = 600,000 achieves the highest reward gain of 771.35 and the greatest wait time reduction of 8.597%, and is therefore selected as the final configuration for all subsequent experiments. The results suggest that a higher epsilon decay generally produces better performance under asymmetric demand, whereas the higher learning rate of 1e-3 consistently outperforms 3e-4 across both the tested decay settings.

Table 1: Full Policy Comparison (50 Episodes, Symmetric Demand). DQN row highlighted.

Policy	Avg Reward	Avg Wait	Avg Queue	Avg Departed	Avg Switches
Fixed-Time (Keep)	-9944.87	31.5431	5.9916	8649.0	0.0
Fixed-Time (Demand-Aware)	-9944.87	31.5431	5.9916	8649.0	0.0
Fixed-Time (Tuned-Static)	-9944.87	31.5431	5.9916	8649.0	0.0
Fixed-Time (Static a=0)	-10509.38	33.3745	6.1229	8650.1	1.0
Fixed-Time (Static a=1)	-9944.87	31.5431	5.9916	8649.0	0.0
Fixed-Time (Static a=2)	-10485.71	33.2987	6.1106	8647.8	1.0
Random Policy	-10393.07	32.8015	6.0824	8649.0	201.0
Queue-Based Policy	-10277.71	32.5231	6.0440	8649.7	110.3
DQN Policy	**-9863.72**	**31.1727**	**5.9066**	**8649.3**	**133.1**

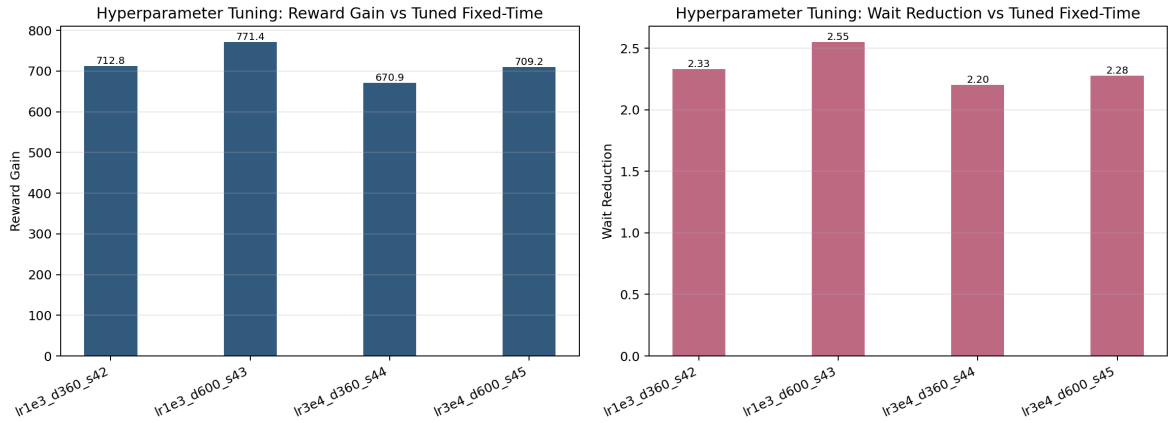
3.2 Pilot Hyperparameter Tuning (Single Seed, Asymmetric-Peaky)

Table 2 and Figure 4 show results of the pilot hyperparameter search on asymmetric-peaky demand. The configuration lr=1e-3 and epsilon_decay=600,000 achieves the highest reward gain of 771.35 and the highest wait reduction of 8.597%, and was selected as the final configuration for all subsequent experiments. The results indicate that a longer exploration schedule (higher epsilon decay) consistently improves performance under asymmetric demand, and that the higher learning rate of 1e-3 outperforms 3e-4 across both decay values tested.

Table 2: Pilot Hyperparameter Search (Single Seed, Asymmetric-Peaky Demand)

LR / Eps Decay	Reward Gain	Wait Diff	Wait Reduction (%)
lr=1e-3, decay=360000	712.85	2.332	7.861%
lr=1e-3, decay=600000	771.35	2.551	8.597%
lr=3e-4, decay=360000	670.88	2.204	7.427%
lr=3e-4, decay=600000	709.24	2.276	7.670%

Figure 4: Hyperparameter tuning comparison — reward gain and wait reduction vs. tuned fixed-time



3.3 Ablation Study: Training Duration (n=4 Seeds)

Table 3 presents the duration ablation results averaged over four seeds under symmetric demand. Training for 1800 episodes achieves a slightly higher mean reward gain (60.14 vs. 52.42) and a marginally higher wait time reduction (0.827% vs. 0.793%) compared to 600 episodes. However, the 1800-episode setting exhibits considerably higher variance across seeds (standard deviation of 87.23 vs. 25.46 for reward gain), making the difference statistically

unreliable. This indicates that under symmetric demand, longer training provides only limited average improvement while introducing greater instability.

Table 3: Duration Ablation Results (mean ± std, 95% CI in parentheses, n=4).

Config	Reward Gain vs Tuned	Wait Diff vs Tuned	Wait Reduction (%)
ep600	52.42 ± 25.46 (±24.95)	0.249 ± 0.101 (±0.099)	0.793%
ep1800	60.14 ± 87.23 (±85.49)	0.260 ± 0.311 (±0.305)	0.827%

3.4 Ablation Study: Model Size (n=4 Seeds)

Table 4 presents the model-size ablation results under symmetric demand. The hidden64 configuration substantially outperforms hidden128 in both mean performance and training stability. The hidden128 model shows extreme inter-seed variance (std of 138.43 in reward gain and 0.473 in wait difference), indicating that the larger output-layer bottleneck is prone to unstable training in this environment. The hidden64 architecture is therefore the recommended configuration for this observation space and action dimensionality.

Table 4: Model Size Ablation Results (mean ± std, 95% CI in parentheses, n=4)

Config	Reward Gain vs Tuned	Wait Diff vs Tuned	Wait Reduction (%)
hidden64	106.62 ± 44.23 (±43.35)	0.424 ± 0.169 (±0.165)	1.347%
hidden128	1.94 ± 138.43 (±135.66)	0.076 ± 0.473 (±0.464)	0.240%

3.5 Final Results: Symmetric vs. Asymmetric-Peaky Demand (n=5 Seeds)

Table 5 and Figure 5 present the final main results comparing symmetric and asymmetric-peaky demand conditions using the selected configuration (hidden_dim=64, lr=1e-3, epsilon_decay=600,000), averaged over five seeds. Under symmetric demand, DQN achieves a reward gain of 115.87 and a wait reduction of 1.455%. Under asymmetric-peaky demand, the reward gain increases to 706.00 and the wait reduction increases to 7.741%. In both scenarios the tuned-static baseline selects action 1 (avg_tuned_action=1.00), confirming that no static action can match DQN performance and that all DQN gains arise from dynamic cycle-level adaptation rather than from identifying a better fixed action. Figure 4 shows the policy-level comparison on the best tuned run, illustrating DQN's advantage in both reward and average wait time across all baselines.

The large gap between the two conditions — a roughly five-fold difference in wait reduction — has a direct structural explanation. Under symmetric demand, all four lanes receive equal average traffic volume (arrival probability 0.30 each). In this setting, a fixed-time policy that splits green time equally between NS and EW is already near-optimal because there is no persistent directional imbalance for the agent to exploit. The DQN can make marginal cycle-to-cycle improvements, but the headroom above the best fixed-time policy is inherently small. Under asymmetric-peaky demand, by contrast, the NS and EW approaches receive systematically different traffic volumes (0.40 and 0.20 for NS, 0.35 and 0.15 for EW), and demand fluctuates with higher amplitude and a shorter period. A fixed-time policy cannot respond to this imbalance and consistently under-serves the congested direction. The DQN learns to dynamically shift green time toward whichever direction is currently more congested,

providing a structural advantage that compounds over each cycle. This is precisely the scenario where adaptive signal control is most valuable, and the results confirm this directly.

Table 5: Final Results by Demand Condition (mean ± std, 95% CI in parentheses, n=5)

Scenario	Reward Gain vs Tuned	Wait Diff vs Tuned	Wait Reduction (%)	Tuned Action
Symmetric	115.87 ± 56.92 (±49.89)	0.458 ± 0.227 (±0.199)	1.455%	1.00
Asymmetric-Peaky	706.00 ± 49.85 (±43.70)	2.292 ± 0.182 (±0.159)	7.741%	1.00

Figure 4: Main difficulty results — symmetric vs. asymmetric-peaky demand

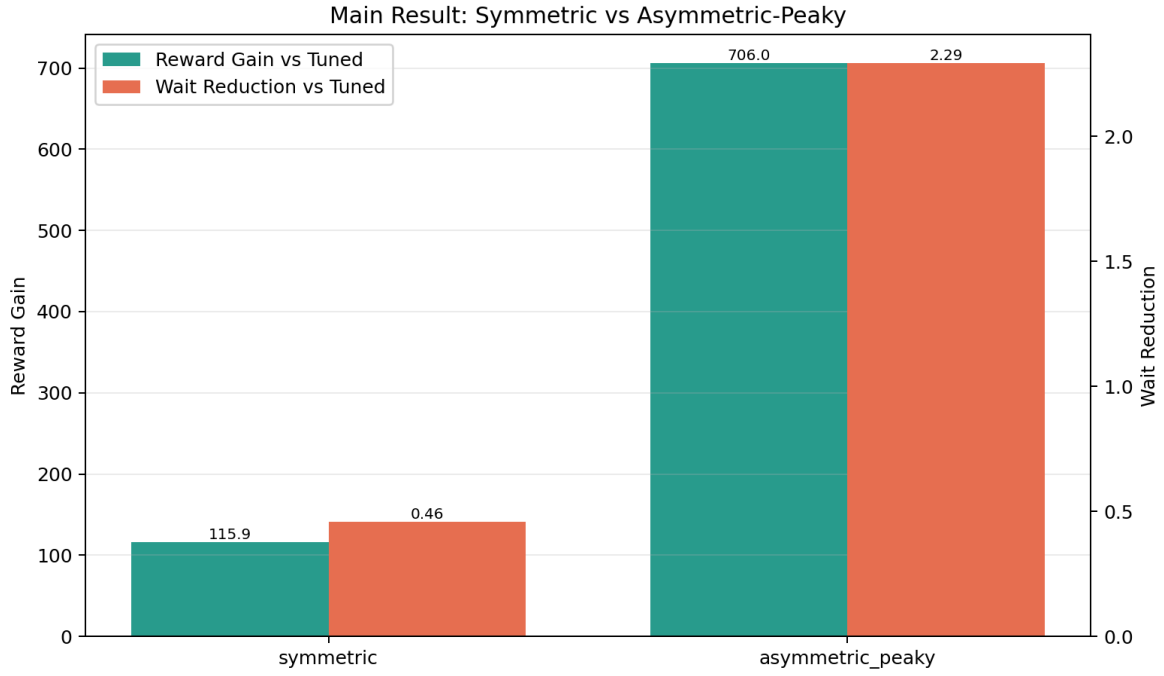
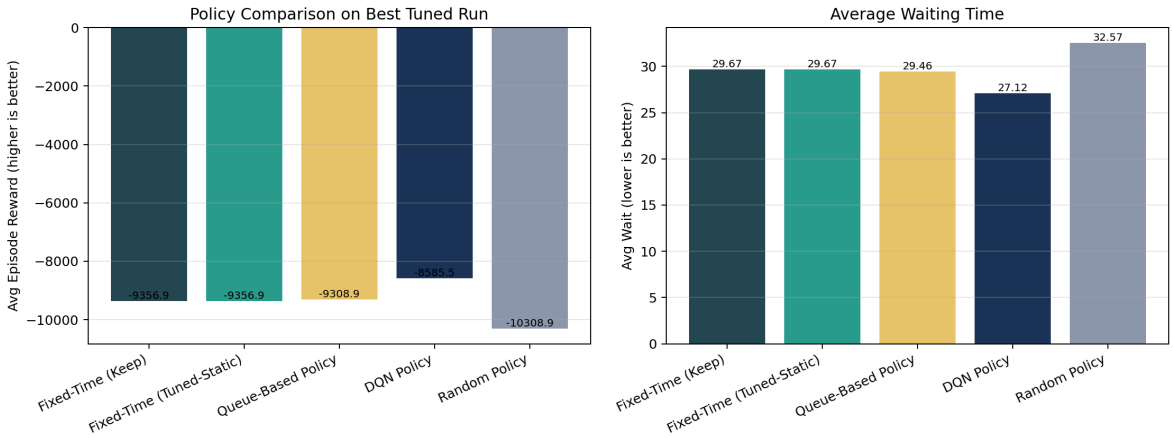


Figure 5: Policy comparison on best tuned run — reward and average wait.



3.6 Symmetric Ceiling Diagnostics (n=5 Seeds)

To investigate whether modest symmetric gains reflect a fundamental ceiling or hyperparameter sensitivity, an additional sweep evaluated six configurations under symmetric demand with n=5 seeds and 100 evaluation episodes per seed. The six settings varied epsilon decay (180,000, 240,000, or 600,000 steps) and the action-switch penalty weight (0.05, 0.15, or 0.30). Table 6 presents the results.

Table 6: Symmetric Ceiling Sweep (mean ± std, 95% CI in parentheses, n=5).

Setting	Reward Gain vs Tuned	Wait Diff vs Tuned	Wait Reduction (%)	Avg Switch Count
d240_sw015	105.84 ± 53.51 (±46.90)	0.365 ± 0.185 (±0.163)	1.158%	90.94
base	99.45 ± 55.50 (±48.64)	0.409 ± 0.210 (±0.184)	1.297%	113.75
d180_sw015	88.31 ± 53.34 (±46.76)	0.302 ± 0.182 (±0.159)	0.957%	102.45
d180	42.26 ± 51.56 (±45.19)	0.189 ± 0.188 (±0.165)	0.600%	71.79
d180_sw005	-23.15 ± 234.15 (±205.24)	-0.078 ± 0.759 (±0.665)	-0.248%	77.54
d240	-24.42 ± 248.96 (±218.22)	-0.014 ± 0.835 (±0.732)	-0.044%	89.40

The best-performing symmetric configurations (d240_sw015 and base) yield wait reductions of approximately 1.0% to 1.3%, substantially smaller than the 7.741% observed under asymmetric-peaky demand. Two configurations (d240 and d180_sw005) produce negative reward gains and near-zero or slightly negative wait differences, indicating that these hyperparameter choices are unstable and can cause DQN to underperform the tuned-static baseline. Overall, two conclusions are supported: symmetric demand is close to a performance ceiling for this control structure, but hyperparameter stability still matters and some settings reliably achieve small positive gains while others do not.

4. Conclusions

This project designed and evaluated a cycle-level DQN controller for single-intersection traffic signal control, aligned with the method skeleton of Moreno-Malo et al. (2024) [1]. The agent consistently outperforms all evaluated baseline policies across both symmetric and asymmetric demand conditions, with the magnitude of improvement scaling strongly with the degree of demand asymmetry.

The contrast between symmetric and asymmetric results reflects a fundamental property of adaptive signal control. Under symmetric demand, all lanes receive equal average traffic, so a balanced fixed-time policy is already close to optimal and leaves little room for improvement. The DQN still achieves small positive gains by responding to short-term fluctuations, but the ceiling is low. Under asymmetric-peaky demand, persistent directional imbalance means fixed-time policies consistently waste green time on lower congested approaches. The DQN learns to reallocate green time dynamically toward a direction which is currently congested, compounding savings across every cycle. This is where adaptive control provides the most value,

and the roughly five-fold difference in wait reduction between the two conditions — 1.455% versus 7.741% — directly quantifies that advantage. The implication for deployment is clear: cycle-level DQN control is the most beneficial in intersections with unequal or time-varying directional demand, and its advantage reduces in settings that closely resemble the symmetric, balanced assumption underlying fixed-time design.

The primary limitations of this work are threefold. First, the simulator is a custom single-intersection environment rather than a multi-intersection platform such as SUMO. The control structure is intentionally designed for compatibility with future SUMO migration, but current results apply only to single-intersection settings. Second, the pilot hyperparameter search is performed on a single seed, so tuning conclusions are directional rather than statistically definitive. Third, some ablation conditions, particularly ep1800 and hidden128, exhibit high inter-seed variance that warrants further investigation.

The strongest result in this study is observed under asymmetric-peaky demand: DQN improves reward by approximately 706 points and reduces the average wait time metric by 2.292 units, corresponding to a 7.741% reduction versus the tuned-static fixed-time baseline. Under symmetric demand, additional ceiling diagnostics confirm that gains of approximately 1.0% to 1.3% in wait reduction are achievable under stable hyperparameter settings, consistent with a near-ceiling regime where there is little directional imbalance for the agent to exploit.

Future work could extend this implementation to a multi-intersection SUMO environment, explore Double DQN or Dueling DQN architectures to address Q-value overestimation, and apply the method to real-world traffic count data rather than synthetic stochastic arrivals.

5. Works Cited

[1] J. Moreno-Malo, J.-L. Posadas-Yagüe, J. C. Cano, C. T. Calafate, J. A. Conejero, and J.-L. Poza-Lujan, *"Improving traffic light systems using Deep Q-networks,"* Expert Systems with Applications, vol. 252, Art. 124178, May 2024. https://doi.org/10.1016/j.eswa.2024.124178

[2] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, et al., *"Human-level control through deep reinforcement learning,"* Nature, vol. 518, pp. 529–533, 2015.

[3] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, et al., *"PyTorch: An imperative style, high-performance deep learning library,"* Advances in Neural Information Processing Systems, vol. 32, 2019.

[4] M. Towers, J. K. Terry, A. Kwiatkowski, J. U. Balis, G. de Cola, T. Deleu, et al., *"Gymnasium,"* Zenodo, 2023. https://doi.org/10.5281/zenodo.8127026